IaC Confluent Cloud AWS Ingress Private Linking, Infrastructure with KMS and Networking Example

Officially, on **February 13, 2026**, Confluent announced support for **Ingress Gateway** and **Ingress Gateway Access Points** as the new standard resources for establishing private connectivity between **AWS** and Confluent Cloud. These capabilities supersede legacy **PrivateLink Attachment (PLATT)** resources and PLATT-based connections moving forward.

> **Note:** Support for PLATT resources will be deprecated in a future release.

This repository delivers a comprehensive, production-grade **Terraform** reference implementation for building a fully private connectivity architecture between Amazon Web Services (AWS) and Confluent Cloud using **AWS Ingress PrivateLink**. It demonstrates how to implement a centralized DNS strategy using **Route 53 Private Hosted Zones** and **AWS Transit Gateway** to enable secure, scalable, multi-VPC access to Confluent Cloud Kafka clusters—without exposing traffic to the public internet. Additionally, it provisions **AWS KMS** customer-managed keys for **Bring Your Own Key (BYOK)** encryption, ensuring that all data at rest in Confluent Cloud Kafka clusters is encrypted with keys you own and control.

The architecture models enterprise-ready patterns, including:

- Centralized Private Hosted Zone (PHZ) management
- Multi-VPC PrivateLink interface endpoint connectivity
- Transit Gateway–based hub-and-spoke routing
- Strict network isolation with no public ingress/egress paths
- **AWS KMS BYOK encryption** for Confluent Cloud Kafka cluster data at rest

Below is the Terraform resource visualization of the infrastructure:

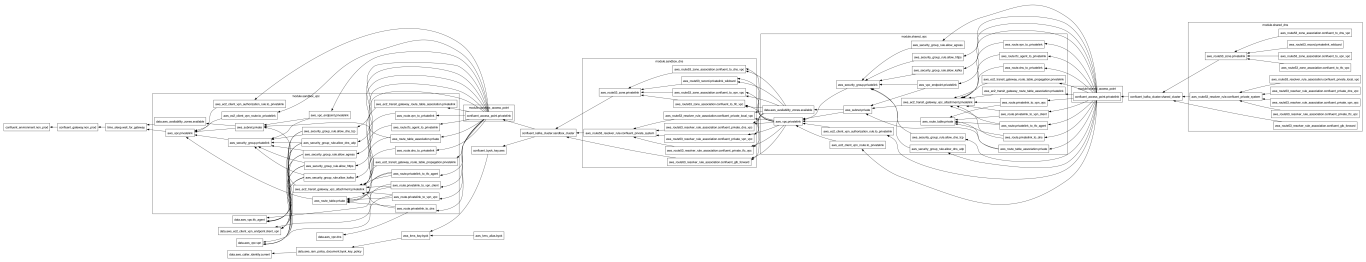

Table of Contents

- 1.0 Prerequisites
  - 1.1 Client VPN, Centralized DNS Server, and Transit Gateway
    - 1.1.1 Key Features Required for Confluent PrivateLink to Work
      - 1.1.1.1 Hub-and-Spoke Network Architecture via Transit Gateway
      - 1.1.1.2 Centralized DNS Resolution (Critical for Ingress PrivateLink)
      - 1.1.1.3 DNS Forwarding Chain
      - 1.1.1.4 VPC Endpoints (AWS Ingress PrivateLink)
      - 1.1.1.5 Client VPN Integration
      - 1.1.1.6 Cross-VPC Routing

- 1.1.1.7 Security & Observability
- 1.2 Terraform Cloud Agent
  - 1.2.1 Key Features of the TFC Agent Setup
    - 1.2.1.1 Custom DHCP Options for DNS Resolution
    - 1.2.1.2 Transit Gateway Connectivity
    - 1.2.1.3 Security Group Configuration for Kafka/Ingress PrivateLink Traffic
    - 1.2.1.4 AWS VPC Endpoints for Private Service Access
    - 1.2.1.5 ECS Fargate Deployment Pattern
    - 1.2.1.6 IAM Permissions for Infrastructure Management
    - 1.2.1.7 Network Architecture Summary
- 2.0 Project's Architecture Overview
  - 2.1 Why This Architecture?
    - 2.1.1 The Problem: Ingress PrivateLink Is VPC-Scoped, But Your Organization Isn't
    - 2.1.2 The Solution: Centralized DNS with a Single PHZ and Smart CNAMEs
    - 2.1.3 The Critical Piece Most Architectures Miss: The SYSTEM Resolver Rule
    - 2.1.4 Why Not VPC Peering?
    - 2.1.5 Why Separate VPCs Per Cluster Instead of One Big VPC?
    - 2.1.6 The Terraform Cloud Agent Piece
    - 2.1.7 AWS KMS BYOK Encryption
- 3.0 Let's Get Started
  - 3.1 Deploy the Infrastructure
  - 3.2 Teardown the Infrastructure
- 4.0 References
  - 4.1 Terminology
  - 4.2 Related Documentation

1.0 Prerequisites

This project assumes you have the following prerequisites in place:

- Client VPN, Centralized DNS Server, and Transit Gateway
- Terraform Cloud Agent

1.1 Client VPN, Centralized DNS Server, and Transit Gateway

```
---
title: AWS Client VPN + Transit Gateway Integration — signalroom-iac-aws-
client-vpn-tgw-integration
---
flowchart TB
    %% ═══ External Actor
═══════════════════════════════════════════════════

    subgraph REMOTE["🌐  Remote User"]
        CLIENT["💻  VPN Client\n(AWS VPN Client App)\n.ovpn config"]
    end

    %% ═══ Client VPN VPC
```

```
────────────────────────────────────────────────────────────
    subgraph VPNVPC["Client VPN VPC  (vpc_cidr)"]
        direction TB

        subgraph ENDPOINT["AWS Client VPN Endpoint"]

VPNEP["aws_ec2_client_vpn_endpoint\n──────────────────────\nclient_cidr_blo
ck (VPN
pool)\nserver_certificate_arn\nauthentication_type\nsplit_tunnel\ntranspor
t_protocol / vpn_port\ndns_servers → dns_vpc_resolver_ips"]
            AUTHRULE["aws_ec2_client_vpn_authorization_rule\n(per target
CIDR)"]
            VPNROUTE["aws_ec2_client_vpn_route\n(per subnet × per workload
CIDR)"]
            ASSOC["aws_ec2_client_vpn_network_association\n(per subnet)"]
        end

        subgraph SUBNETS["Subnets (1 per AZ)"]
            SN1["Subnet AZ-a\ncidrsubnet(vpc_cidr, bits, 0)"]
            SN2["Subnet AZ-b\ncidrsubnet(vpc_cidr, bits, 1)"]
            SNN["Subnet AZ-n …"]
        end

        subgraph ROUTETABLES["Route Tables (per AZ)"]
            RT1["RTB AZ-a\n→ workload CIDRs via TGW\n→ dns_vpc_cidr via
TGW\n→ vpn_client_cidr via VPN ENI"]
            RT2["RTB AZ-b"]
        end

        subgraph RESOLVER["Route53 Outbound Resolver"]
            RESOLVERSG["aws_security_group\n(resolver-outbound-sg)\nEgress
53/TCP+UDP → dns_vpc_cidr"]
            RESOLVEREP["aws_route53_resolver_endpoint\n(OUTBOUND, 2
subnets)"]

RESRULE["aws_route53_resolver_rule\n(FORWARD)\nconfluent_private_domain\n→
dns_vpc_resolver_ips"]
            RESRULEADD["aws_route53_resolver_rule\n(FORWARD —
additional_private_domains)"]
            RESASSOC["aws_route53_resolver_rule_association\nvpn_vpc"]
        end

        VPNSG["aws_security_group (client-vpn-sg)\nIngress:
workload_vpc_cidrs\nEgress: 0.0.0.0/0"]
        FLOWLOG["aws_flow_log (optional)\n+ aws_iam_role vpc_flow_logs\n+
aws_cloudwatch_log_group /aws/vpc/flow-logs"]
    end

    %% ─── Transit Gateway
────────────────────────────────────────────────────────────
    subgraph TGW["Transit Gateway (existing — tgw_id)"]
        TGWATT["aws_ec2_transit_gateway_vpc_attachment\n(client_vpn VPC)"]
        TGWRTA["aws_ec2_transit_gateway_route_table_association\n→
tgw_rt_id"]
```

```
        TGWRTP["aws_ec2_transit_gateway_route_table_propagation\nvpc_cidr
→ tgw_rt_id"]
        TGWRTS["Static Routes in tgw_rt_id\n→ vpc_cidr\n→
vpn_client_cidr"]
        TGWRTBL["TGW Route Table (tgw_rt_id)"]
    end

    %% ─── DNS VPC
──────────────────────────────────────────────────────────────────────
    subgraph DNSVPC["DNS VPC (existing)"]
        INBOUNDRES["Route53 Inbound Resolver\n(dns_vpc_resolver_ips)"]
        PHZ["Private Hosted Zones\n(Confluent PrivateLink domains)"]
        VPCE["VPC Endpoints\n(Confluent PrivateLink ENIs)"]
    end

    %% ─── Workload VPCs
──────────────────────────────────────────────────────────────────
    subgraph WORKVPCS["Workload VPCs (existing — workload_vpc_cidrs)"]
        WV1["Workload VPC 1\n(Kafka / Flink services)"]
        WV2["Workload VPC 2 …"]
    end

    %% ─── Confluent Cloud
──────────────────────────────────────────────────────────────────
    subgraph CONFLUENT["☁ Confluent Cloud (private.confluent.cloud)"]
        CC["Confluent Kafka / Schema Registry\n(accessed via
PrivateLink)"]
    end

    %% ─── Observability
──────────────────────────────────────────────────────────────────
    subgraph OBS["CloudWatch / IAM"]
        CWG["aws_cloudwatch_log_group (VPN
connections)\naws_cloudwatch_log_stream"]
        IAMTGW["aws_iam_role tgw_flow_logs\n(vpc-flow-
logs.amazonaws.com)"]
        IAMVPC["aws_iam_role vpc_flow_logs\n(vpc-flow-
logs.amazonaws.com)"]
    end

    %% ─── Terraform Cloud
──────────────────────────────────────────────────────────────────
    subgraph TFC["Terraform Cloud"]
        WS["workspace: signalroom-iac-aws-client-vpn-tgw-integration\norg:
signalroom\nexecution_mode: local"]
    end

    %% ─── Connections
──────────────────────────────────────────────────────────────────────

    %% VPN client → endpoint
    CLIENT -- "TLS/UDP\n(transport_protocol/vpn_port)" --> VPNEP
    VPNEP --> VPNSG
    VPNEP --> ASSOC
```

```
    ASSOC --> SN1
    ASSOC --> SN2
    VPNEP --> AUTHRULE
    VPNEP --> VPNROUTE
    VPNEP -- "connection logs" --> CWG

    %% Subnets → Route Tables
    SN1 --> RT1
    SN2 --> RT2

    %% Route Tables → TGW
    RT1 -- "workload CIDRs\n+ dns_vpc_cidr" --> TGWATT
    RT2 -- "workload CIDRs\n+ dns_vpc_cidr" --> TGWATT

    %% VPN VPC → TGW attachment
    VPNVPC --> TGWATT
    TGWATT --> TGWRTA
    TGWATT --> TGWRTP
    TGWATT --> TGWRTS
    TGWRTA --> TGWRTBL
    TGWRTP --> TGWRTBL
    TGWRTS --> TGWRTBL

    %% TGW → downstream VPCs
    TGWRTBL --> DNSVPC
    TGWRTBL --> WORKVPCS

    %% DNS resolution chain
    VPNEP -- "dns_servers=dns_vpc_resolver_ips" --> RESOLVEREP
    RESOLVEREP --> RESOLVERSG
    RESRULE --> RESOLVEREP
    RESRULEADD --> RESOLVEREP
    RESASSOC --> RESRULE
    RESOLVEREP -- "port 53 via TGW" --> INBOUNDRES
    INBOUNDRES --> PHZ
    PHZ -- "private IP" --> VPCE
    VPCE <--> CC

    %% Workload access
    CLIENT -- "data plane\n(via TGW after DNS resolves)" --> WORKVPCS

    %% Flow logs / IAM
    FLOWLOG --> IAMVPC
    IAMTGW -.-> TGW
    CWG --> OBS

    %% Terraform Cloud
    TFC -.-> VPNVPC
    TFC -.-> TGW

    %% ─── Styles
```

```
    classDef aws fill:#FF9900,color:#000,stroke:#232F3E,stroke-width:1.5px
    classDef dns fill:#7AA116,color:#fff,stroke:#232F3E,stroke-width:1.5px
```

```
    classDef tgw fill:#E7157B,color:#fff,stroke:#232F3E,stroke-width:1.5px
    classDef confluent fill:#0074E4,color:#fff,stroke:#003087,stroke-
width:1.5px
    classDef obs fill:#4B0082,color:#fff,stroke:#232F3E,stroke-width:1.5px
    classDef tfc fill:#5C4EE5,color:#fff,stroke:#4040BB,stroke-width:1.5px
    classDef client fill:#1A9C3E,color:#fff,stroke:#0E5C24,stroke-
width:1.5px

    class VPNEP,VPNSG,ASSOC,AUTHRULE,VPNROUTE,SN1,SN2,SNN,RT1,RT2,FLOWLOG
aws
    class
RESOLVEREP,RESOLVERSG,RESRULE,RESRULEADD,RESASSOC,INBOUNDRES,PHZ,VPCE dns
    class TGWATT,TGWRTA,TGWRTP,TGWRTS,TGWRTBL tgw
    class CC,CONFLUENT confluent
    class CWG,IAMTGW,IAMVPC obs
    class WS,TFC tfc
    class CLIENT client
```

1.1.1 Key Features Required for Confluent Ingress PrivateLink to Work

1.1.1.1 Hub-and-Spoke Network Architecture via Transit Gateway

- Transit Gateway serves as the central routing hub connecting all VPCs
- Disabled default route table association/propagation for explicit routing control
- DNS support enabled on the TGW (`dns_support = "enable"`)
- Custom route tables for fine-grained traffic control between VPCs

1.1.1.2 Centralized DNS Resolution (Critical for Ingress PrivateLink)

- **Dedicated DNS VPC** with Route53 Inbound Resolver endpoints
- DNS forwarding rules route Confluent queries from all VPCs to the central DNS VPC
- Route53 Outbound Resolver in VPN VPC forwards to DNS VPC resolver IPs

1.1.1.3 DNS Forwarding Chain (as documented in your outputs)

1. VPN VPC's default DNS forwards to Route53 Outbound Resolver
2. Outbound Resolver forwards to DNS VPC Inbound Resolver
3. DNS VPC checks Private Hosted Zones → returns VPC Endpoint private IPs

1.1.1.4 VPC Endpoints (AWS Ingress PrivateLink)

- VPC Endpoints in workload VPCs connecting to Confluent's Ingress PrivateLink service
- Security groups allowing traffic from authorized sources (VPN clients, TFC agents)

1.1.1.5 Client VPN Integration

- Mutual TLS authentication using ACM certificates (server + client)
- Split tunnel configuration for routing only Confluent traffic through VPN
- Authorization rules controlling which CIDRs VPN clients can access

- Routes added to VPN endpoint for all workload VPC CIDRs via Transit Gateway

1.1.1.6 Cross-VPC Routing

- TGW attachments for: VPN VPC, DNS VPC, TFC Agent VPC, and all Workload VPCs
- Route tables in each VPC with routes to other VPCs via TGW
- Workload VPC CIDRs aggregated and distributed to VPN client routes

1.1.1.7 Security & Observability

- Dedicated security groups per component (VPN endpoint, etc.)
- VPC Flow Logs and TGW Flow Logs to CloudWatch
- VPN connection logging for audit trails
- IAM roles with least-privilege for flow log delivery

1.2 Terraform Cloud Agent

```
%%{init: {'theme': 'base', 'themeVariables': { 'primaryColor': '#1a73e8',
'primaryTextColor': '#fff', 'primaryBorderColor': '#1557b0', 'lineColor':
'#5f6368', 'secondaryColor': '#34a853', 'tertiaryColor': '#fbbc04'}}}%%

flowchart TB
    subgraph TERRAFORM_CLOUD["☁ Terraform Cloud (HCP)"]
        TFC["Terraform Cloud
API & Workspaces"]
        AgentPool["Agent Pool
(signalroom)"]
    end

    subgraph AWS["☁ AWS Cloud"]
        subgraph TFC_AGENT_VPC["TFC Agent VPC
var.vpc_cidr"]
            subgraph PUBLIC_SUBNETS["Public Subnets (Multi-AZ)"]
                IGW["Internet
Gateway"]
                NAT1["NAT Gateway
AZ-1"]
                NAT2["NAT Gateway
AZ-2"]
            end

            subgraph PRIVATE_SUBNETS["Private Subnets (Multi-AZ)"]
                subgraph ECS["ECS Fargate Cluster"]
                    TFCAgent1["TFC Agent
Container"]
                    TFCAgent2["TFC Agent
Container"]
                end

                subgraph AWS_ENDPOINTS["AWS VPC Endpoints"]
                    VPCE_SM["Secrets Manager
```

```
Endpoint"]
                    VPCE_CW["CloudWatch Logs
Endpoint"]
                    VPCE_ECR["ECR API/DKR
Endpoints"]
                    VPCE_S3["S3 Gateway
Endpoint"]
                end

                CONFLUENT_SG["Confluent PrivateLink
Security Group"]
            end

            DHCP["DHCP Options
(Custom DNS)"]
            TFC_AGENT_SG["TFC Agent
Security Group"]
        end

        subgraph TGW["Transit Gateway
signalroom-tgw"]
            TGWCore["TGW Core"]
            TGWRT["Route Table"]
        end

        subgraph DNS_VPC["DNS VPC (Centralized)
var.dns_vpc_cidr"]
            R53Inbound["Route53 Inbound
Resolver"]
            PHZ["Private Hosted Zones
*.aws.confluent.cloud"]
        end

        subgraph CLIENT_VPN_VPC["Client VPN VPC
var.client_vpn_vpc_cidr"]
            VPNEndpoint["Client VPN
Endpoint"]
        end

        subgraph WORKLOAD_VPCs["Workload VPCs
(Confluent PrivateLink)"]
            subgraph WL1["Workload VPC 1"]
                VPCE1["PrivateLink
Endpoint"]
            end
            subgraph WL2["Workload VPC N"]
                VPCEN["PrivateLink
Endpoint"]
            end
        end

        SecretsManager["AWS Secrets Manager
(TFC Agent Token)"]
        CloudWatch["CloudWatch Logs"]
```

```
        ECR_Registry["ECR Registry
(hashicorp/tfc-agent)"]
    end

    subgraph CONFLUENT["☁ Confluent Cloud"]
        PrivateLinkSvc["PrivateLink
Service"]
        Kafka["Kafka Cluster
(Private)"]
    end

    %% External Connections
    TFC <-->|"HTTPS/443
via NAT"| TFCAgent1
    TFC <-->|"HTTPS/443
via NAT"| TFCAgent2
    AgentPool -.->|"Agent Registration"| TFCAgent1

    %% Internal VPC Connections
    TFCAgent1 --> TFC_AGENT_SG
    TFCAgent2 --> TFC_AGENT_SG
    TFCAgent1 --> VPCE_SM
    TFCAgent2 --> VPCE_CW

    VPCE_SM -.->|"Private DNS"| SecretsManager
    VPCE_CW -.->|"Private DNS"| CloudWatch
    VPCE_ECR -.->|"Private DNS"| ECR_Registry

    NAT1 --> IGW
    NAT2 --> IGW
    TFCAgent1 -->|"0.0.0.0/0"| NAT1
    TFCAgent2 -->|"0.0.0.0/0"| NAT2

    %% DHCP & DNS Flow
    DHCP -->|"DNS Servers:
VPC + Centralized"| TFCAgent1
    TFCAgent1 -->|"DNS Query:
*.confluent.cloud"| R53Inbound

    %% Transit Gateway Connections
    TFC_AGENT_VPC -->|"TGW Attachment"| TGW
    DNS_VPC -->|"TGW Attachment"| TGW
    CLIENT_VPN_VPC -->|"TGW Attachment"| TGW
    WL1 -->|"TGW Attachment"| TGW
    WL2 -->|"TGW Attachment"| TGW

    %% Route Propagation
    TGWCore --> TGWRT

    %% DNS Resolution
    R53Inbound --> PHZ
    PHZ -->|"Returns Private IPs"| VPCE1

    %% PrivateLink Connections
```

```
    VPCE1 -->|"AWS PrivateLink"| PrivateLinkSvc
    VPCEN -->|"AWS PrivateLink"| PrivateLinkSvc
    PrivateLinkSvc --> Kafka

    %% TFC Agent to Workload VPCs
    TFC_AGENT_SG -->|"HTTPS/443
Kafka/9092"| CONFLUENT_SG
    CONFLUENT_SG -->|"via TGW"| VPCE1
    CONFLUENT_SG -->|"via TGW"| VPCEN

    %% Styling
    classDef tfcStyle fill:#5c4ee5,stroke:#3d32a8,stroke-
width:2px,color:#fff
    classDef vpcStyle fill:#e8f0fe,stroke:#1a73e8,stroke-width:2px
    classDef tgwStyle fill:#fef7e0,stroke:#f9ab00,stroke-width:3px
    classDef dnsStyle fill:#e6f4ea,stroke:#34a853,stroke-width:2px
    classDef confluentStyle fill:#f3e8fd,stroke:#9334e6,stroke-width:2px
    classDef endpointStyle fill:#fce8e6,stroke:#ea4335,stroke-width:1px
    classDef ecsStyle fill:#fff3e0,stroke:#ff9800,stroke-width:2px

    class TERRAFORM_CLOUD tfcStyle
    class TFC_AGENT_VPC,CLIENT_VPN_VPC,WORKLOAD_VPCs,WL1,WL2 vpcStyle
    class TGW tgwStyle
    class DNS_VPC dnsStyle
    class CONFLUENT confluentStyle
    class AWS_ENDPOINTS,VPCE_SM,VPCE_CW,VPCE_ECR,VPCE_S3 endpointStyle
    class ECS ecsStyle
```

1.2.1 Key Features Required for Confluent PrivateLink to Work (TFC Agent Configuration)

1.2.1.1 Custom DHCP Options for DNS Resolution

- DHCP Options Set configured with **dual DNS servers**: VPC default DNS (`cidrhost(vpc_cidr, 2)`) AND centralized DNS VPC resolver IPs
- Region-aware domain name configuration (`ec2.internal` for us-east-1, `{region}.compute.internal` for others)
- Associates TFC Agent VPC with custom DHCP options to route Confluent domain queries to the central DNS infrastructure

1.2.1.2 Transit Gateway Connectivity

- TFC Agent VPC attached to shared Transit Gateway with DNS support enabled
- Explicit route table association and route propagation (not using TGW defaults)
- Routes added from private subnets to: DNS VPC, Client VPN VPC, and all Workload VPCs containing PrivateLink endpoints
- Flattened route map pattern (`for_each`) ensures routes are created for every workload VPC CIDR

1.2.1.3 Security Group Configuration for Kafka/Ingress PrivateLink Traffic

- **TFC Agent Security Group** with egress rules for:

    - HTTPS (443) and Kafka (9092) to each workload VPC CIDR
    - DNS (UDP/TCP 53) to DNS VPC CIDR specifically
    - General HTTPS/HTTP for Terraform Cloud API and package downloads
- **Confluent Ingress PrivateLink Security Group** allowing inbound from TFC Agent SG on ports 443 and 9092

1.2.1.4 AWS VPC Endpoints for Private Service Access

- **Interface endpoints** with private DNS enabled for: Secrets Manager, CloudWatch Logs, ECR API, ECR DKR
- **S3 Gateway endpoint** (required for ECR image layer pulls)
- Dedicated security group for VPC endpoints allowing HTTPS from within VPC
- Eliminates NAT Gateway dependency for AWS service calls

1.2.1.5 ECS Fargate Deployment Pattern

- TFC Agents run in private subnets with `assign_public_ip = false`
- NAT Gateways per AZ for outbound internet (Terraform Cloud API communication)
- Agent token stored in Secrets Manager, fetched via VPC Endpoint
- Container health checks and deployment circuit breaker for reliability

1.2.1.6 IAM Permissions for Infrastructure Management

- Task role with Transit Gateway, VPC, Route53 Resolver, and Client VPN management permissions
- Execution role with Secrets Manager access for agent token retrieval
- KMS permissions scoped to Secrets Manager service for encryption/decryption

1.2.1.7 Network Architecture Summary

- **Hub-and-spoke model**: TGW connects TFC Agent VPC → DNS VPC → Workload VPCs
- **DNS resolution chain**: TFC Agent → Custom DHCP → Centralized DNS VPC → Private Hosted Zones → PrivateLink Endpoint IPs
- **Traffic flow**: TFC Agent → TGW → Workload VPC → PrivateLink Endpoint → Confluent Cloud Kafka

2.0 Project's Architecture Overview

This repo creates a multi-VPC architecture where Confluent Cloud Enterprise Kafka clusters are reachable exclusively over private network path that never traverses the public internet.

```
graph TB
    subgraph CC["☁️ Confluent Cloud (non-prod environment)"]
        GW["🔀 PrivateLink Gateway\nnon-prod-privatelink-gateway"]
        BYOK["🔑 BYOK Key\n(confluent_byok_key)"]
        subgraph SBcluster["Sandbox Kafka Cluster\n(Enterprise · HIGH
availability)"]
            SBAP["Access Point\nccloud-accesspoint-sandbox"]
        end
        subgraph SHcluster["Shared Kafka Cluster\n(Enterprise · HIGH
```

```
availability)"]
            SHAP["Access Point\nccloud-accesspoint-shared"]
        end
        GW --> SBAP
        GW --> SHAP
        BYOK -.->|"encrypts data at rest"| SBcluster
        BYOK -.->|"encrypts data at rest"| SHcluster
    end

    subgraph KMS["🔐 AWS KMS"]
        KMSKEY["Customer-Managed KMS Key\nalias/confluent-cloud-
byok\n(auto-rotation enabled)"]
    end

    subgraph TGW["🔁 AWS Transit Gateway"]
        TGWRT["TGW Route Table\n(associations + propagations)"]
    end

    subgraph SB_VPC["🟦 Sandbox PrivateLink VPC\n10.0.0.0/20"]
        SB_SN["Private Subnets ×3\n(multi-AZ)"]
        SB_EP["VPC Endpoint\n(Interface · PrivateLink)"]
        SB_SG["Security Group\nports 443, 9092, 53/UDP, 53/TCP"]
        SB_RT["Route Tables ×3"]
        SB_EP --> SB_SG
        SB_SN --> SB_EP
        SB_SN --> SB_RT
    end

    subgraph SH_VPC["🟩 Shared PrivateLink VPC\n10.1.0.0/20"]
        SH_SN["Private Subnets ×3\n(multi-AZ)"]
        SH_EP["VPC Endpoint\n(Interface · PrivateLink)"]
        SH_SG["Security Group\nports 443, 9092, 53/UDP, 53/TCP"]
        SH_RT["Route Tables ×3"]
        SH_EP --> SH_SG
        SH_SN --> SH_EP
        SH_SN --> SH_RT
    end

    subgraph VPN_VPC["🔐 VPN VPC"]
        CVPN["AWS Client VPN Endpoint\n(VPN routes + auth rules)"]
        DEVS["🧑‍💻 Developer / Remote Clients"]
        DEVS -->|"VPN tunnel"| CVPN
    end

    subgraph TFC_VPC["⚙ TFC Agent VPC"]
        TFCA["Terraform Cloud Agent\n(ECS Fargate)"]
    end

    subgraph DNS_VPC["🌐 DNS VPC"]
        R53R["Route53 Resolver\n(Inbound / Outbound)"]
    end

    subgraph DNS_SB["Route53 — Sandbox DNS"]
        SB_PHZ["Private Hosted Zone\n(access point domain)"]
```

```
        SB_WILD["Wildcard CNAME *.domain\n→ VPC Endpoint DNS"]
        SB_SYSRULE["Resolver SYSTEM Rule\n(domain → local PHZ)"]
        SB_GLBFWD["Resolver FWD Rule\n(Confluent global domain)"]
        SB_PHZ --> SB_WILD
    end

    subgraph DNS_SH["Route53 — Shared DNS"]
        SH_PHZ["Private Hosted Zone\n(access point domain)"]
        SH_WILD["Wildcard CNAME *.domain\n→ VPC Endpoint DNS"]
        SH_SYSRULE["Resolver SYSTEM Rule\n(domain → local PHZ)"]
        SH_GLBFWD["Resolver FWD Rule\n(Confluent global domain)"]
        SH_PHZ --> SH_WILD
    end

    %% PrivateLink connectivity
    SB_EP <-->|"AWS PrivateLink"| SBAP
    SH_EP <-->|"AWS PrivateLink"| SHAP

    %% TGW attachments
    SB_SN <-->|"TGW attachment"| TGW
    SH_SN <-->|"TGW attachment"| TGW
    VPN_VPC <-->|"TGW routes"| TGW
    TFC_VPC <-->|"TGW routes"| TGW
    DNS_VPC <-->|"TGW routes"| TGW

    %% Route cross-references via TGW
    SB_RT -->|"→ TFC Agent CIDR via TGW"| TGW
    SB_RT -->|"→ VPN CIDR via TGW"| TGW
    SB_RT -->|"→ DNS CIDR via TGW"| TGW
    SH_RT -->|"→ TFC Agent CIDR via TGW"| TGW
    SH_RT -->|"→ VPN CIDR via TGW"| TGW
    SH_RT -->|"→ DNS CIDR via TGW"| TGW

    %% PHZ associations
    SB_PHZ -.->|"PHZ assoc"| SB_VPC
    SB_PHZ -.->|"PHZ assoc"| DNS_VPC
    SB_PHZ -.->|"PHZ assoc"| VPN_VPC
    SB_PHZ -.->|"PHZ assoc"| TFC_VPC
    SH_PHZ -.->|"PHZ assoc"| SH_VPC
    SH_PHZ -.->|"PHZ assoc"| DNS_VPC
    SH_PHZ -.->|"PHZ assoc"| VPN_VPC
    SH_PHZ -.->|"PHZ assoc"| TFC_VPC

    %% Resolver rule associations
    SB_SYSRULE -.->|"assoc"| DNS_VPC
    SB_SYSRULE -.->|"assoc"| VPN_VPC
    SB_SYSRULE -.->|"assoc"| TFC_VPC
    SB_SYSRULE -.->|"assoc"| SB_VPC
    SB_GLBFWD -.->|"assoc"| SB_VPC
    SH_SYSRULE -.->|"assoc"| DNS_VPC
    SH_SYSRULE -.->|"assoc"| VPN_VPC
    SH_SYSRULE -.->|"assoc"| TFC_VPC
    SH_SYSRULE -.->|"assoc"| SH_VPC
    SH_GLBFWD -.->|"assoc"| SH_VPC
```

```
    %% TFC / Infra provisioning
    TFCA -->|"provision via Terraform Cloud"| SB_VPC
    TFCA -->|"provision via Terraform Cloud"| SH_VPC

    %% KMS BYOK encryption
    KMSKEY -->|"cross-account access"| BYOK

    classDef confluent fill:#0073e6,color:#fff,stroke:#005bb5
    classDef aws fill:#FF9900,color:#000,stroke:#cc7a00
    classDef vpn fill:#2e7d32,color:#fff,stroke:#1b5e20
    classDef dns fill:#6a1b9a,color:#fff,stroke:#4a148c
    classDef tfc fill:#5c4ee5,color:#fff,stroke:#3d35b5
    classDef tgw fill:#bf360c,color:#fff,stroke:#870000
    classDef kms fill:#d32f2f,color:#fff,stroke:#b71c1c

    class GW,SBAP,SHAP,BYOK confluent
    class SB_EP,SB_SG,SH_EP,SH_SG aws
    class CVPN,DEVS vpn
    class
SB_PHZ,SB_WILD,SB_SYSRULE,SB_GLBFWD,SH_PHZ,SH_WILD,SH_SYSRULE,SH_GLBFWD,R5
3R dns
    class TFCA tfc
    class TGW,TGWRT tgw
    class KMSKEY kms
```

2.1 Why This Architecture?

Confluent Cloud PrivateLink connectivity introduces three interconnected challenges that most naive implementations fail to solve. This architecture addresses all three systematically.

2.1.1 The Problem: PrivateLink Is VPC-Scoped, But Your Organization Isn't

AWS PrivateLink creates an interface VPC endpoint inside a single VPC. The endpoint gets private IP addresses within that VPC's CIDR range, and DNS resolution to the Confluent cluster's bootstrap and broker endpoints must resolve to those private IPs. This creates an immediate problem: what about all the other VPCs in your AWS environment that also need to reach Confluent?

In a typical enterprise setup you have infrastructure VPCs (for CI/CD agents, DNS, VPN gateways) that all need to reach the same Kafka clusters. Without a deliberate cross-VPC strategy, you'd need to duplicate PrivateLink endpoints and DNS configuration in every single VPC — an operational and cost nightmare that doesn't scale.

2.1.2 The Solution: Centralized Transit Gateway as the Network Backbone

This architecture uses AWS Transit Gateway as a centralized routing hub that connects all VPCs. Each PrivateLink VPC (Sandbox and Shared) attaches to the Transit Gateway, and bidirectional routes are established between the PrivateLink VPCs and every infrastructure VPC that needs access (TFC Agent VPC, DNS VPC, VPN VPC). This means any workload in any attached VPC can route traffic to the PrivateLink endpoint's private IPs through the Transit Gateway, without needing its own endpoint.

The key insight is that the VPC endpoint only needs to exist in one place per cluster, but the routes to reach it can be propagated across the entire Transit Gateway topology. This is what makes the architecture scale: adding a new VPC that needs Confluent access is just a Transit Gateway attachment and a few route entries, not a full PrivateLink setup.

2.1.3 The DNS Challenge: Why This Is Harder Than It Looks

This is where most Confluent PrivateLink implementations get tricky. Confluent's Kafka clusters use DNS-based routing extensively — the bootstrap server resolves to a hostname, which returns broker-specific hostnames, which must resolve to availability-zone-specific endpoints for proper data locality. All of this DNS resolution must return the private IP addresses of the VPC endpoint, not the public Confluent IPs.

This architecture solves the DNS challenge with three layers:

1. **Centralized Private Hosted Zone (PHZ)**: A single Route 53 PHZ is created for the Confluent PrivateLink DNS domain and associated with all VPCs that need resolution. This eliminates PHZ duplication and ensures consistent DNS answers everywhere.
2. **Zonal and Wildcard CNAME Records**: The PHZ contains availability-zone-specific CNAME records (e.g., `*.use1-az1.domain → vpce-xxx-use1-az1.vpce-svc.amazonaws.com`) that ensure Kafka clients connect to brokers in their local AZ, preserving data locality and minimizing cross-AZ data transfer costs. A wildcard record handles the bootstrap endpoint.
3. **SYSTEM Resolver Rule**: This is the critical piece most architectures miss. In complex AWS environments, Route 53 Resolver may have FORWARD rules that send DNS queries to on-premises or external DNS servers. These FORWARD rules can intercept Confluent domain queries before the PHZ is consulted, breaking PrivateLink resolution entirely. The SYSTEM resolver rule explicitly tells Route 53 Resolver: "For this specific Confluent domain, resolve locally using the PHZ — do not forward anywhere." This rule is associated with every VPC in the architecture, providing a safety net against DNS forwarding conflicts.

2.1.4 Why Not VPC Peering?

VPC Peering is a valid alternative to Transit Gateway for simple topologies, but it doesn't scale well for this use case. Peering is non-transitive (if VPC-A peers with VPC-B, and VPC-B peers with VPC-C, VPC-A cannot reach VPC-C through VPC-B). With five VPCs that all need to reach two PrivateLink VPCs, you'd need a mesh of peering connections that becomes unwieldy. Transit Gateway provides transitive routing through a single hub, keeping the topology clean and the route table management centralized.

2.1.5 Why Separate VPCs Per Cluster Instead of One Big VPC?

Each Kafka cluster gets its own VPC and PrivateLink endpoint through the reusable `aws-vpc-confluent-privatelink` module. This provides network-level isolation between environments (sandbox vs. shared), independent CIDR management, independent security group policies per cluster, and the ability to tear down one cluster's networking without affecting others. The module pattern also means adding a third, fourth, or fifth cluster follows the exact same playbook.

2.1.6 The Terraform Cloud Agent Piece

The architecture runs Terraform Cloud in agent execution mode, where TFC Agents run inside a private VPC within AWS. This is essential because the Terraform provider must be able to reach the Confluent PrivateLink endpoints to validate connections and manage resources. If Terraform ran in the default remote execution mode (on HashiCorp's infrastructure), it wouldn't have network access to the private endpoints. By running agents in a VPC that's attached to the Transit Gateway and associated with the centralized PHZ, Terraform can resolve and reach the PrivateLink endpoints during plan and apply operations.

2.1.7 AWS KMS BYOK Encryption

This architecture implements **Bring Your Own Key (BYOK)** encryption for Confluent Cloud Kafka clusters using **AWS Key Management Service (KMS)**. Instead of relying on Confluent-managed encryption keys, you provision and control a customer-managed KMS key in your own AWS account, giving you full ownership over the encryption lifecycle for data at rest.

The KMS integration works as follows:

1. **Key Provisioning**: A customer-managed KMS key is created in your AWS account (or you can supply an existing key ARN via the `aws_kms_key_arn` Terraform variable). The key is aliased as `alias/confluent-cloud-byok` and configured with automatic key rotation enabled and a 14-day deletion protection window.
2. **Cross-Account Access Policy**: The KMS key policy grants the Confluent Cloud AWS account (identified by `confluent_byok_account_id`) the minimum permissions required for BYOK encryption: `Encrypt`, `Decrypt`, `ReEncrypt*`, `GenerateDataKey*`, `DescribeKey`, `CreateGrant`, `ListGrants`, and `RevokeGrant`. Your AWS account root retains full `kms:*` management access.
3. **BYOK Registration**: The KMS key is registered with Confluent Cloud through the `confluent_byok_key` resource, which establishes the trust relationship between your AWS KMS key and Confluent's encryption infrastructure.
4. **Cluster Encryption**: Both the Sandbox and Shared Kafka clusters are configured with a `byok_key` block referencing the registered BYOK key, ensuring all data at rest is encrypted using your customer-managed key.

This approach satisfies compliance requirements (e.g., SOC 2, HIPAA, PCI-DSS) that mandate customer-controlled encryption keys, and gives you the ability to revoke Confluent's access to the key at any time — effectively crypto-shredding all cluster data.

3.0 Let's Get Started

3.1 Deploy the Infrastructure

The deploy.sh script handles authentication and Terraform execution:

```
./deploy.sh create --profile=<SSO_PROFILE_NAME> \
                   --confluent-api-key=<CONFLUENT_API_KEY> \
                   --confluent-api-secret=<CONFLUENT_API_SECRET> \
                   --tgw-id=<TGW_ID> \
                   --tgw-rt-id=<TGW_RT_ID> \
                   --tfc-agent-vpc-id=<TFC_AGENT_VPC_ID> \
                   --tfc-agent-vpc-rt-ids=<TFC_AGENT_VPC_RT_IDs> \
                   --dns-vpc-id=<DNS_VPC_ID> \
```

```
                      --dns-vpc-rt-ids=<DNS_VPC_RT_IDs> \
                      --vpn-vpc-id=<VPN_VPC_ID> \
                      --vpn-vpc-rt-ids=<VPN_VPC_RT_IDs> \
                      --vpn-endpoint-id=<VPN_ENDPOINT_ID> \
                      --vpn-target-subnet-ids=<VPN_TARGET_SUBNET_IDs> \
                      --confluent-glb-resolver-rule-id=
<CONFLUENT_GLB_RESOLVER_RULE_ID>
```

Here's the argument table for `deploy.sh create` command:

Argument	Required	Description
`--profile`	✅	The AWS SSO profile name. Passed directly to `aws sso login` and `aws2-wrap` for authentication, and used to resolve `AWS_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN`, which are then exported as `TF_VAR_aws_region`, `TF_VAR_aws_access_key_id`, `TF_VAR_aws_secret_access_key`, and `TF_VAR_aws_session_token` for Terraform, respectively.
`--confluent-api-key`	✅	Confluent Cloud API key. Exported as `TF_VAR_confluent_api_key` for Terraform.
`--confluent-api-secret`	✅	Confluent Cloud API secret. Exported as `TF_VAR_confluent_api_secret` for Terraform.
`--tgw-id`	✅	AWS Transit Gateway ID. Exported as `TF_VAR_tgw_id` for routing between VPCs.
`--tgw-rt-id`	✅	AWS Transit Gateway Route Table ID. Exported as `TF_VAR_tgw_rt_id` for associating route entries.
`--tfc-agent-vpc-id`	✅	VPC ID where the Terraform Cloud Agent resides. Exported as `TF_VAR_tfc_agent_vpc_id`.
`--tfc-agent-vpc-rt-ids`	✅	Route table IDs for the TFC Agent VPC (supports multiple, unquoted).
`--dns-vpc-id`	✅	VPC ID for the DNS resolver infrastructure. Exported as `TF_VAR_dns_vpc_id`.
`--dns-vpc-rt-ids`	✅	Route table IDs for the DNS VPC (supports multiple, unquoted). Exported as `TF_VAR_dns_vpc_rt_ids`.
`--vpn-vpc-id`	✅	VPC ID for the VPN infrastructure. Exported as `TF_VAR_vpn_vpc_id`.
`--vpn-vpc-rt-ids`	✅	Route table IDs for the VPN VPC (supports multiple, unquoted). Exported as `TF_VAR_vpn_vpc_rt_ids`.

Argument	Required	Description
`--vpn-endpoint-id`	✅	AWS Client VPN endpoint ID. Exported as `TF_VAR_vpn_endpoint_id`.
`--vpn-target-subnet-ids`	✅	Subnet IDs associated with the VPN endpoint target network. Exported as `TF_VAR_vpn_target_subnet_ids`.
`--confluent-glb-resolver-rule-id`	✅	The ID of the SYSTEM resolver rule in Route 53 that ensures Confluent domain queries are resolved locally within AWS and not forwarded to external DNS servers. Exported as `TF_VAR_confluent_glb_resolver_rule_id`.

> All 15 arguments are required — the script exits with code `85` if any are missing.

Additional Terraform Variables for KMS BYOK Encryption

The following variables must be set separately (e.g., via `terraform.tfvars`, environment variables, or Terraform Cloud workspace variables) as they are not passed through the `deploy.sh` script:

Variable	Required	Description
`confluent_byok_account_id`	✅	The Confluent Cloud AWS account ID that will be granted cross-account access to use the KMS key for BYOK encryption.
`aws_kms_key_arn`	❌	Optional existing AWS KMS key ARN for Confluent Cloud BYOK encryption. If not provided, a new KMS key will be created with automatic key rotation enabled.

3.2 Teardown the Infrastructure

```
./deploy.sh destroy --profile=<SSO_PROFILE_NAME> \
                    --confluent-api-key=<CONFLUENT_API_KEY> \
                    --confluent-api-secret=<CONFLUENT_API_SECRET> \
                    --tgw-id=<TGW_ID> \
                    --tgw-rt-id=<TGW_RT_ID> \
                    --tfc-agent-vpc-id=<TFC_AGENT_VPC_ID> \
                    --tfc-agent-vpc-rt-ids=<TFC_AGENT_VPC_RT_IDs> \
                    --dns-vpc-id=<DNS_VPC_ID> \
                    --dns-vpc-rt-ids=<DNS_VPC_RT_IDs> \
                    --vpn-vpc-id=<VPN_VPC_ID> \
                    --vpn-vpc-rt-ids=<VPN_VPC_RT_IDs> \
                    --vpn-endpoint-id=<VPN_ENDPOINT_ID> \
                    --vpn-target-subnet-ids=<VPN_TARGET_SUBNET_IDs> \
                    --confluent-glb-resolver-rule-id=
<CONFLUENT_GLB_RESOLVER_RULE_ID>
```

Here's the argument table for `deploy.sh destroy` command:

Argument	Required	Description
`--profile`	✅	The AWS SSO profile name. Passed directly to `aws sso login` and `aws2-wrap` for authentication, and used to resolve `AWS_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN`, which are then exported as `TF_VAR_aws_region`, `TF_VAR_aws_access_key_id`, `TF_VAR_aws_secret_access_key`, and `TF_VAR_aws_session_token` for Terraform, respectively.
`--confluent-api-key`	✅	Confluent Cloud API key. Exported as `TF_VAR_confluent_api_key` for Terraform.
`--confluent-api-secret`	✅	Confluent Cloud API secret. Exported as `TF_VAR_confluent_api_secret` for Terraform.
`--tgw-id`	✅	AWS Transit Gateway ID. Exported as `TF_VAR_tgw_id` for routing between VPCs.
`--tgw-rt-id`	✅	AWS Transit Gateway Route Table ID. Exported as `TF_VAR_tgw_rt_id` for associating route entries.
`--tfc-agent-vpc-id`	✅	VPC ID where the Terraform Cloud Agent resides. Exported as `TF_VAR_tfc_agent_vpc_id`.
`--tfc-agent-vpc-rt-ids`	✅	Route table IDs for the TFC Agent VPC (supports multiple, unquoted).
`--dns-vpc-id`	✅	VPC ID for the DNS resolver infrastructure. Exported as `TF_VAR_dns_vpc_id`.
`--dns-vpc-rt-ids`	✅	Route table IDs for the DNS VPC (supports multiple, unquoted). Exported as `TF_VAR_dns_vpc_rt_ids`.
`--vpn-vpc-id`	✅	VPC ID for the VPN infrastructure. Exported as `TF_VAR_vpn_vpc_id`.
`--vpn-vpc-rt-ids`	✅	Route table IDs for the VPN VPC (supports multiple, unquoted). Exported as `TF_VAR_vpn_vpc_rt_ids`.
`--vpn-endpoint-id`	✅	AWS Client VPN endpoint ID. Exported as `TF_VAR_vpn_endpoint_id`.

Argument	Required	Description
`--vpn-target-subnet-ids`	✅	Subnet IDs associated with the VPN endpoint target network. Exported as `TF_VAR_vpn_target_subnet_ids`.
`--confluent-glb-resolver-rule-id`	✅	The ID of the SYSTEM resolver rule in Route 53 that ensures Confluent domain queries are resolved locally within AWS and not forwarded to external DNS servers. Exported as `TF_VAR_confluent_glb_resolver_rule_id`.

> All 15 arguments are required — the script exits with code `85` if any are missing.

4.0 Resources

4.1 Terminology

- **PHZ**: Private Hosted Zone - AWS Route 53 Private Hosted Zone is a DNS service that allows you to create and manage private DNS zones within your VPCs.
- **TFC**: Terraform Cloud - A service that provides infrastructure automation using Terraform.
- **VPC**: Virtual Private Cloud - A virtual network dedicated to your AWS account.
- **AWS**: Amazon Web Services - A comprehensive cloud computing platform provided by Amazon.
- **CC**: Confluent Cloud - A fully managed event streaming platform based on Apache Kafka.
- **PL**: PrivateLink - An AWS service that enables private connectivity between VPCs and services.
- **IaC**: Infrastructure as Code - The practice of managing and provisioning computing infrastructure through machine-readable definition files.
- **KMS**: Key Management Service - AWS service for creating and managing cryptographic keys used to encrypt data.
- **BYOK**: Bring Your Own Key - An encryption model where customers provide and control their own encryption keys rather than using provider-managed keys.

4.2 Related Documentation

- Confluent Cloud BYOK Encryption
- Confluent Cloud BYOK for AWS
- AWS PrivateLink Overview in Confluent Cloud
- Use AWS PrivateLink for Serverless Products on Confluent Cloud
- GitHub Sample Project for Confluent Terraform Provider PrivateLink Attachment
- Use the Confluent Cloud Console with Private Networking
- IP Filtering on Confluent Cloud
- AWS/Azure PrivateLink Networking Course
- Hands On: Configuring a PrivateLink Cluster